PRSECURITY HRMS — Full Build Prompt for Nvidia Nemotron

ROLE & OBJECTIVE

You are a senior full-stack engineer. Your task is to build a complete, production-ready **HRMS (Human Resource Management System)** web application for a cybersecurity company called **PRSECURITY CONSULTANCY & SERVICES** based in Surat, India.

This is an **internal-only office management platform** — not a SaaS product, no multi-tenancy, no payment gateway. All data must persist permanently in a PostgreSQL database. Restarting the application must never cause data loss.

TECH STACK — MANDATORY, DO NOT DEVIATE

Frontend

React 18 with **Vite** and **TypeScript**
Tailwind CSS for all styling
shadcn/ui for all UI components (buttons, modals, forms, tables, tabs, cards, dropdowns, toasts)
React Router v6 for routing
React Hook Form + **Zod** for all form validation
Recharts for all charts and analytics
Axios for all API calls
Socket.io-client for real-time notifications
date-fns for date handling
react-pdf or **jsPDF** for PDF payslip generation on client side

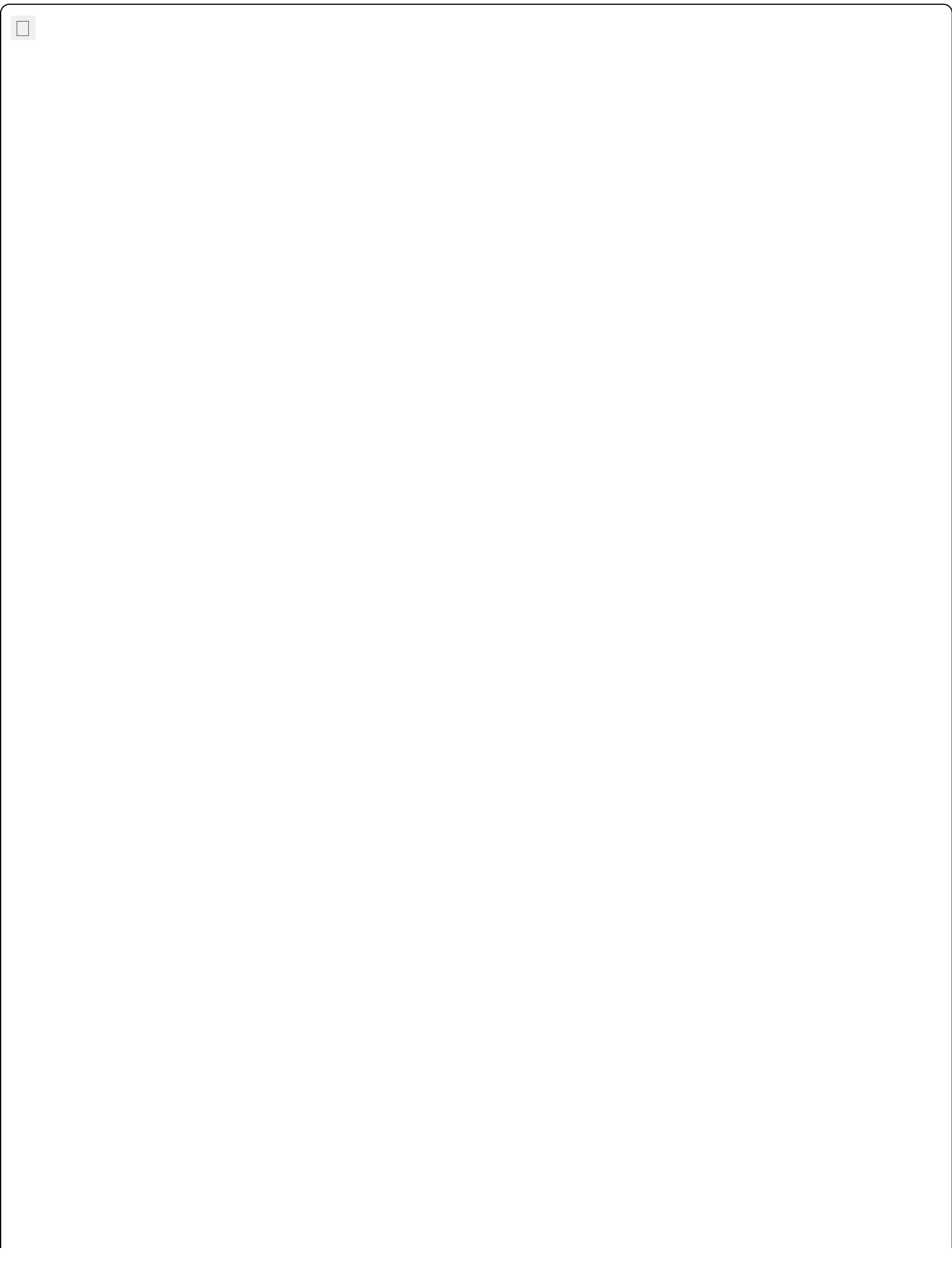
Backend

Node.js with **Express.js** and **TypeScript**
Prisma ORM with **PostgreSQL** as the database
JWT (jsonwebtoken) for authentication — access token (15 min) + refresh token (7 days) stored in httpOnly cookies
bcryptjs for password hashing
Socket.io for real-time notifications
Multer for file uploads (documents, profile pictures)
node-cron for scheduled tasks (monthly payroll generation trigger, leave balance reset)
cors, **helmet**, **express-rate-limit** for security middleware
dotenv for environment variables

Database

PostgreSQL (latest stable)
Managed via **Prisma** with full schema and migrations
All seeds included for initial Admin account

PROJECT STRUCTURE



```
orsecurity-hrms/
├── frontend/
│   ├── src/
│   │   ├── api/              # Axios instances and all API call functions
│   │   ├── components/       # Reusable UI components
│   │   │   ├── layout/       # Sidebar, Topbar, Layout wrapper
│   │   │   ├── ui/           # shadcn/ui components
│   │   │   └── shared/       # DataTable, StatCard, PageHeader, etc.
│   │   ├── hooks/            # Custom React hooks (useAuth, useSocket, etc.)
│   │   ├── pages/            # One folder per module
│   │   │   ├── auth/
│   │   │   ├── dashboard/
│   │   │   ├── employees/
│   │   │   ├── attendance/
│   │   │   ├── leave/
│   │   │   ├── payroll/
│   │   │   ├── tasks/
│   │   │   ├── projects/
│   │   │   ├── clients/
│   │   │   ├── recruitment/
│   │   │   ├── performance/
│   │   │   ├── assets/
│   │   │   ├── helpdesk/
│   │   │   ├── announcements/
│   │   │   └── reports/
│   │   ├── store/            # Zustand global state (auth, notifications)
│   │   ├── types/            # TypeScript interfaces for all entities
│   │   ├── utils/            # Helper functions (formatCurrency, formatDate, etc.)
│   │   ├── routes/           # Protected route wrappers per role
│   │   └── App.tsx
│   ├── index.html
│   ├── vite.config.ts
│   ├── tailwind.config.ts
│   └── tsconfig.json
│
├── backend/
│   ├── src/
│   │   ├── controllers/      # One controller per module
│   │   ├── routes/           # Express routers per module
│   │   ├── middleware/       # auth, roleGuard, errorHandler, upload
│   │   ├── services/         # Business logic layer
│   │   ├── utils/            # JWT helpers, payroll calculator, email sender
│   │   ├── jobs/             # node-cron scheduled jobs
│   │   ├── socket/           # Socket.io event handlers
```

```
│   │   └── app.ts          # Express app setup
│   ├── prisma/
│   │   ├── schema.prisma   # Full database schema
│   │   ├── migrations/     # All Prisma migrations
│   │   └── seed.ts         # Seed: default Admin account
│   ├── uploads/            # Multer storage (profile pics, docs)
│   ├── .env.example
│   └── tsconfig.json
│
├── docker-compose.yml      # PostgreSQL + pgAdmin container
└── README.md               # Full setup instructions
```

DATABASE SCHEMA (Prisma)

Build the complete `schema.prisma` with all the following models, relations, and constraints:

User

```
id, employeeId (unique, auto-generated e.g. PRS-001), firstName, lastName,
email (unique), passwordHash, role (ADMIN | HR | EMPLOYEE | INTERN),
phone, profilePicture, dateOfBirth, dateOfJoining, designation,
employmentType (FULL_TIME | PART_TIME | INTERN | CONTRACT),
status (ACTIVE | INACTIVE | ON_LEAVE | TERMINATED),
address, emergencyContactName, emergencyContactPhone,
bankAccountNumber, bankIFSC, bankName, panNumber,
createdAt, updatedAt
```

SalaryStructure

```
id, userId (unique FK → User), basicSalary, hra, travelAllowance,
medicalAllowance, otherAllowances, pfEmployeePercent (default 12),
pfEmployerPercent (default 12), esiEmployeePercent (default 0.75),
esiEmployerPercent (default 3.25), tdsPercent, createdAt, updatedAt
```

Attendance

```
id, userId (FK → User), date, punchIn, punchOut,
punchInLat, punchInLong, punchOutLat, punchOutLong,
punchInIP, punchOutIP, workingHours (computed),
status (PRESENT | ABSENT | HALF_DAY | HOLIDAY | WEEKEND | ON_LEAVE),
notes, isManualEntry (bool), approvedBy (FK → User nullable),
createdAt, updatedAt
Unique: (userId, date)
```

LeaveType

□

id, name (e.g. "Casual Leave"), code (CL, SL, EL, CO, ML, etc.),
defaultDays (annual allocation), isPaid (bool), description, createdAt

LeaveBalance

□

id, userId (FK), leaveTypeId (FK), year, allocated, used, remaining
Unique: (userId, leaveTypeId, year)

LeaveRequest

□

id, userId (FK → User), leaveTypeId (FK), fromDate, toDate,
totalDays, reason, status (PENDING | APPROVED | REJECTED | CANCELLED),
reviewedBy (FK → User nullable), reviewedAt, reviewerNote,
createdAt, updatedAt

Payroll

□

id, userId (FK → User), month (1–12), year,
basicSalary, hra, travelAllowance, medicalAllowance, otherAllowances,
bonus, deductions (other deductions), pfEmployee, pfEmployer,
esiEmployee, esiEmployer, tds, grossSalary, netSalary,
paymentStatus (PENDING | PROCESSING | PAID), paidAt,
generatedBy (FK → User), payslipUrl, workingDays, presentDays,
createdAt, updatedAt
Unique: (userId, month, year)

Project

□

id, name, description, clientId (FK → Client), status (PLANNING | ACTIVE | ON_HOLD | COMPLETED |
CANCELLED),
startDate, endDate, budget, currency (default INR),
priority (LOW | MEDIUM | HIGH | CRITICAL), createdBy (FK → User),
createdAt, updatedAt

ProjectMember

□

id, projectId (FK), userId (FK), role (LEAD | MEMBER | REVIEWER), joinedAt
Unique: (projectId, userId)

Client

□

id, name, email, phone, address, industry, contactPersonName,
contactPersonEmail, contactPersonPhone, gstin, website,
status (ACTIVE | INACTIVE), createdBy (FK → User), createdAt, updatedAt

Task

id, title, description, projectId (FK → Project nullable),
assignedTo (FK → User), assignedBy (FK → User),
priority (LOW | MEDIUM | HIGH | CRITICAL),
status (TODO | IN_PROGRESS | IN_REVIEW | DONE | CANCELLED),
dueDate, completedAt, estimatedHours, actualHours,
createdAt, updatedAt

TaskComment

id, taskId (FK), userId (FK), content, createdAt

JobPosting

id, title, description, requirements, responsibilities,
department, location, type (FULL_TIME | PART_TIME | INTERN | CONTRACT),
status (OPEN | CLOSED | ON_HOLD), salaryMin, salaryMax,
openings, closingDate, createdBy (FK → User), createdAt, updatedAt

Applicant

id, jobPostingId (FK), firstName, lastName, email, phone,
resumeUrl, coverLetter, currentCompany, currentDesignation,
noticePeriod, expectedSalary,
status (APPLIED | SHORTLISTED | INTERVIEW_SCHEDULED | OFFERED | HIRED | REJECTED),
notes, createdAt, updatedAt

Interview

id, applicantId (FK), scheduledAt, mode (IN_PERSON | VIDEO | PHONE),
interviewers (FK → User array via InterviewerOnInterview join table),
feedback, rating (1–5), result (PASS | FAIL | ON_HOLD), createdAt

PerformanceGoal

id, userId (FK), title, description, targetDate, weight (percentage),
status (ACTIVE | COMPLETED | MISSED), createdBy (FK → User), createdAt, updatedAt

PerformanceReview

id, userId (FK), reviewerId (FK → User), period (Q1/Q2/Q3/Q4/ANNUAL),
year, selfRating (1–5), managerRating (1–5), selfComments,
managerComments, overallRating, status (DRAFT | SUBMITTED | COMPLETED),
createdAt, updatedAt

Asset

□

id, name, type (LAPTOP | PHONE | MONITOR | PERIPHERAL | OTHER),
serialNumber, assignedTo (FK → User nullable), assignedAt,
status (AVAILABLE | ASSIGNED | MAINTENANCE | RETIRED),
purchaseDate, purchasePrice, condition, notes, createdAt, updatedAt

ExpenseClaim

□

id, userId (FK), title, amount, category, receiptUrl, date,
status (PENDING | APPROVED | REJECTED | PAID),
reviewedBy (FK → User nullable), reviewerNote, createdAt, updatedAt

Announcement

□

id, title, content, priority (LOW | MEDIUM | HIGH | URGENT),
targetRoles (array of Role), postedBy (FK → User),
expiresAt, createdAt, updatedAt

HelpdeskTicket

□

id, title, description, category (IT | HR | ADMIN | OTHER),
priority (LOW | MEDIUM | HIGH | URGENT),
status (OPEN | IN_PROGRESS | RESOLVED | CLOSED),
raisedBy (FK → User), assignedTo (FK → User nullable),
resolution, createdAt, updatedAt, resolvedAt

Notification

□

id, userId (FK), title, message, type, isRead (bool),
link (optional deep link), createdAt

HolidayCalendar

□

id, name, date, type (NATIONAL | OPTIONAL | COMPANY), year, createdAt

AUTHENTICATION SYSTEM

POST `/api/auth/login` — email + password → returns accessToken (JWT, 15 min) + sets httpOnly refreshToken cookie

POST `/api/auth/refresh` — uses httpOnly cookie → returns new accessToken
POST `/api/auth/logout` — clears cookie
GET `/api/auth/me` — returns logged-in user profile
POST `/api/auth/change-password` — old password + new password
POST `/api/auth/forgot-password` — sends reset link via email (nodemailer)
POST `/api/auth/reset-password/:token` — resets password
Role Middleware: Every protected route checks JWT + role. Use a `roleGuard(roles: Role[])` middleware factory.
Default seed account:
Email: `admin@prsecurity.in`
Password: `Admin@1234`
Role: `ADMIN`

MODULES — COMPLETE API + UI SPECIFICATIONS

MODULE 1: EMPLOYEE MANAGEMENT

API Endpoints:

```
GET    /api/employees          → list all (HR/Admin); own profile only (Employee/Intern)
POST   /api/employees          → create new employee (Admin/HR only)
GET    /api/employees/:id      → get full profile
PUT    /api/employees/:id      → update profile
DELETE /api/employees/:id      → soft delete / deactivate (Admin only)
POST   /api/employees/:id/documents → upload document (Multer)
GET    /api/employees/:id/documents → list documents
```

UI Pages:

`/employees` — DataTable with columns: Photo, Name, Employee ID, Designation, Role, Status, Joining Date, Actions
Search by name/ID, filter by role/status, pagination
Export to CSV button
`/employees/new` — Multi-step form:
Step 1: Personal Info (name, DOB, phone, address, emergency contact)
Step 2: Employment Info (designation, role, type, joining date)
Step 3: Salary Structure (basic, HRA, allowances, PF%, ESI%, TDS%)
Step 4: Bank Details (account no, IFSC, bank name, PAN)
Step 5: Documents upload
`/employees/:id` — Full profile view with tabs: Overview, Attendance, Leave, Payslips, Tasks, Documents, Assets
Auto-generate employee ID on creation: `PRS-001`, `PRS-002`, etc.

MODULE 2: ATTENDANCE

API Endpoints:

```
POST   /api/attendance/punch-in    → log punch-in with lat/long/IP (Employee/Intern)
POST   /api/attendance/punch-out   → log punch-out with lat/long/IP (Employee/Intern)
GET    /api/attendance/today       → own today's status
GET    /api/attendance/my          → own monthly attendance log (with filters: month, year)
GET    /api/attendance             → all employees' attendance (HR/Admin)
GET    /api/attendance/:userId     → specific employee attendance
POST   /api/attendance/manual      → HR/Admin manually adds/edits record
GET    /api/attendance/summary     → monthly summary stats (present/absent/late/leaves)
```

Geolocation Logic:

On punch-in/out, frontend captures `navigator.geolocation` lat/long

Backend stores lat, long, and request IP

Display location on attendance record (no hard geofencing, just capture + display)

UI Pages:

`/attendance` — For Employee/Intern: large Punch In / Punch Out button with current time, today's status badge, this month's calendar heatmap showing P/A/L/H

`/attendance/team` — For HR/Admin: table of all employees, today's attendance status in real-time, filter by date, export to CSV

`/attendance/logs` — Monthly log table with date, punch-in, punch-out, working hours, status. HR can click any record to edit.

Attendance calendar uses colour coding: Green=Present, Red=Absent, Yellow=Half Day, Blue=Leave, Grey=Weekend/Holiday

Working hours auto-calculation: punchOut time minus punchIn time, stored in decimal hours.

MODULE 3: LEAVE MANAGEMENT

API Endpoints:

```
GET    /api/leaves/types           → all leave types
POST   /api/leaves/types           → create leave type (Admin/HR)
GET    /api/leaves/balance         → own leave balance for current year
GET    /api/leaves/balance/:userId → specific user balance (HR/Admin)
POST   /api/leaves/apply           → apply for leave (Employee/Intern)
GET    /api/leaves/my              → own leave history
GET    /api/leaves                 → all leave requests (HR/Admin) with filters
PUT    /api/leaves/:id/approve     → HR/Admin approve
PUT    /api/leaves/:id/reject      → HR/Admin reject with reason
PUT    /api/leaves/:id/cancel      → Employee cancel their own pending request
GET    /api/leaves/calendar        → team leave calendar (who's on leave on which dates)
```

Leave Types to seed:

Casual Leave (CL) — 12 days/year — Paid

Sick Leave (SL) — 12 days/year — Paid

Earned/Privilege Leave (EL) — 15 days/year — Paid

Comp-off (CO) — 0 (granted manually) — Paid

Maternity Leave (ML) — 180 days — Paid
Paternity Leave (PL) — 15 days — Paid
Unpaid Leave (UL) — Unlimited — Unpaid
Optional Holiday (OH) — 2 days/year — Paid

Balance Logic:

On year start (Jan 1), LeaveBalance rows are created/reset for all active users via cron job
When leave is approved: deduct from balance. When cancelled after approval: restore.
Carry-forward: Earned Leave carries over (max 30 days), others lapse.

UI Pages:

`/leave` — Employee view: Balance cards (one per leave type showing used/remaining/total), Apply Leave button, own request history table
`/leave/apply` — Form: Leave type dropdown, from date, to date (auto-calculates working days excluding weekends/holidays), reason textarea, submit
`/leave/team` — HR/Admin: all pending requests listed, approve/reject buttons with optional note modal
`/leave/calendar` — Visual monthly calendar showing who's on leave each day (colour-coded by person)

MODULE 4: PAYROLL

API Endpoints:

```
GET    /api/payroll/my              → own payslips list
GET    /api/payroll/my/:month/:year → own payslip detail
POST   /api/payroll/generate        → HR/Admin: generate payroll for a month
GET    /api/payroll                 → HR/Admin: all payroll records with filters
GET    /api/payroll/:userId/:month/:year → specific payslip
PUT    /api/payroll/:id/mark-paid   → mark as paid
GET    /api/payroll/:id/pdf         → download payslip PDF
POST   /api/payroll/:id/bonus       → add bonus to a payslip
```

Payroll Calculation Logic (implement as a service function):

```
Gross Salary = Basic + HRA + Travel Allowance + Medical Allowance + Other Allowances + Bonus

PF (Employee) = Basic × 12%          → deducted from employee
PF (Employer) = Basic × 12%          → company contribution (shown but not deducted)
ESI (Employee) = Gross × 0.75%       → only if Gross ≤ ₹21,000/month
ESI (Employer) = Gross × 3.25%       → only if Gross ≤ ₹21,000/month
TDS = configurable % of Gross        → professional tax / income tax withholding

Net Salary = Gross − PF(Employee) − ESI(Employee) − TDS − Other Deductions

Loss of Pay (LOP) = (Basic / Working Days in Month) × Absent Days
```

Payslip PDF must include:

PRSECURITY CONSULTANCY & SERVICES header with logo placeholder
Employee details: Name, ID, Designation, Month-Year

Earnings table: Basic, HRA, Travel, Medical, Other, Bonus → Gross Total
Deductions table: PF, ESI, TDS, LOP, Other → Total Deductions
Employer contributions section (PF, ESI)
Net Pay in ₹ (in words also)
Working days / Present days
Footer: Generated on [date], Authorized Signatory

UI Pages:

`/payroll` — Employee: list of all my payslips by month, Download PDF button per row

`/payroll/run` — HR/Admin: Select Month + Year → "Generate Payroll" → shows preview table of all employees with calculated amounts → Confirm to save

`/payroll/manage` — HR/Admin: table of all payroll records, filter by month/year/status, bulk "Mark as Paid", export to Excel

`/payroll/:id` — Payslip detail view (same layout as PDF, rendered in browser)

MODULE 5: TASK MANAGEMENT

API Endpoints:

```
GET    /api/tasks                → my tasks (Employee); all tasks (HR/Admin)
POST   /api/tasks                → create task (HR/Admin/Employee can create for self)
GET    /api/tasks/:id            → task detail
PUT    /api/tasks/:id            → update task
DELETE /api/tasks/:id            → delete (Admin/HR or creator)
PUT    /api/tasks/:id/status     → update status (assignee)
POST   /api/tasks/:id/comments   → add comment
GET    /api/tasks/:id/comments   → get comments
GET    /api/tasks/stats          → task stats per user / overall
```

UI Pages:

`/tasks` — Kanban board with 4 columns: To Do | In Progress | In Review | Done. Cards show: title, assignee avatar, priority badge (colour coded), due date, project tag. Drag-and-drop between columns updates status via API.

`/tasks/list` — Table view with all filters: status, priority, assignee, project, due date range

`/tasks/:id` — Task detail drawer/modal: full description, comments thread, time tracking (log hours), attachments, activity log

`/tasks/new` — Create task form: title, description, project (optional), assign to (user dropdown), priority, due date, estimated hours

MODULE 6: PROJECT & CLIENT MANAGEMENT

API Endpoints:

```
```

```
GET    /api/clients              → list clients (HR/Admin)
POST   /api/clients              → create client
GET    /api/clients/:id          → client detail + their projects
PUT    /api/clients/:id          → update
DELETE /api/clients/:id          → deactivate

GET    /api/projects             → all projects (HR/Admin); own projects (Employee)
POST   /api/projects             → create project
GET    /api/projects/:id         → project detail
PUT    /api/projects/:id         → update
DELETE /api/projects/:id         → archive
POST   /api/projects/:id/members  → add team member
DELETE /api/projects/:id/members/:userId → remove member
GET    /api/projects/:id/tasks   → all tasks for this project
GET    /api/projects/stats       → project statistics
```

UI Pages:

`/clients` — Card grid: each card shows client name, industry, contact person, active projects count, status badge

`/clients/:id` — Client detail: contact info, associated projects list, notes timeline

`/clients/new` — Form: company name, GSTIN, industry, contact person details, address

`/projects` — Cards or table toggle: project name, client, team avatars, status badge, progress bar (% tasks done), deadline

`/projects/:id` — Project detail tabs:
Overview: description, client, dates, budget, progress
Team: member list with role, add/remove members
Tasks: embedded Kanban filtered to this project
Timeline: Gantt-like view (simple horizontal bars with date ranges)

`/projects/new` — Form: name, description, client, start/end date, priority, budget, assign team members

MODULE 7: RECRUITMENT / ATS

API Endpoints:

```
□
```

```
GET    /api/jobs              → all job postings
POST   /api/jobs              → create posting (HR/Admin)
GET    /api/jobs/:id          → posting detail with applicants
PUT    /api/jobs/:id          → update posting
PUT    /api/jobs/:id/status   → open/close posting

POST   /api/applicants        → add applicant to a job (HR manually adds)
GET    /api/applicants/:id    → applicant detail
PUT    /api/applicants/:id/status → move pipeline stage
POST   /api/applicants/:id/interview → schedule interview
PUT    /api/interviews/:id    → update interview result/feedback
POST   /api/applicants/:id/hire → convert applicant to employee (creates User)
```

UI Pages:

`/recruitment` — Job postings list: title, department, openings, applicant count, status, actions

`/recruitment/new` — Job posting form: title, description, requirements (rich text), responsibilities, type, salary range, openings, closing date

`/recruitment/:jobId` — Job detail + applicant pipeline:
Kanban board with stages: Applied | Shortlisted | Interview Scheduled | Offered | Hired | Rejected
Each applicant card: name, email, expected salary, applied date

`/recruitment/applicants/:id` — Applicant profile: resume download, interview history, status change, feedback, "Hire" button that auto-creates employee account

MODULE 8: PERFORMANCE MANAGEMENT

API Endpoints:

```
GET    /api/performance/goals/my   → own goals
POST   /api/performance/goals      → create goal (self or manager for reportee)
PUT    /api/performance/goals/:id  → update goal
DELETE /api/performance/goals/:id  → delete

GET    /api/performance/reviews/my → own reviews
POST   /api/performance/reviews    → start review cycle (HR/Admin)
PUT    /api/performance/reviews/:id/self → submit self-rating
PUT    /api/performance/reviews/:id/manager → submit manager rating
GET    /api/performance/reviews    → all reviews (HR/Admin)
```

UI Pages:

`/performance` — My Performance: Goal progress bars, current review cycle status card, last review summary

`/performance/goals` — Goals table: title, target date, weight, status. Add goal button.

`/performance/review/:id` — Review form: self-rating slider (1–5) + comments per goal, overall self-comment. For managers: same form + manager rating fields side by side.

`/performance/manage` — HR/Admin: all review cycles, status overview table, initiate new cycle button

MODULE 9: ASSETS & EXPENSES

API Endpoints:

```
GET    /api/assets              → all assets (HR/Admin)
POST   /api/assets              → add asset
PUT    /api/assets/:id          → update
POST   /api/assets/:id/assign   → assign to employee
POST   /api/assets/:id/return   → mark as returned

GET    /api/expenses/my         → own claims
POST   /api/expenses            → submit claim (with receipt upload)
GET    /api/expenses            → all claims (HR/Admin)
PUT    /api/expenses/:id/approve → approve
PUT    /api/expenses/:id/reject  → reject with reason
PUT    /api/expenses/:id/pay     → mark paid
```

UI Pages:

`/assets` — Asset inventory table: serial no, type, assigned to, status, purchase date. Assign/Return modal.

`/expenses` — Employee: my claims table + Submit New Claim form (title, category, amount ₹, date, receipt upload)

`/expenses/manage` — HR/Admin: all claims, approve/reject buttons, filter by status/employee

MODULE 10: ANNOUNCEMENTS & HELPDESK

API Endpoints:

```
GET    /api/announcements       → active announcements for my role
POST   /api/announcements       → create (HR/Admin)
PUT    /api/announcements/:id   → update
DELETE /api/announcements/:id   → delete/expire

GET    /api/tickets/my          → own tickets
POST   /api/tickets             → raise ticket
GET    /api/tickets             → all tickets (HR/Admin)
PUT    /api/tickets/:id/assign  → assign to HR/Admin
PUT    /api/tickets/:id/resolve → mark resolved
PUT    /api/tickets/:id/close   → close
```

UI Pages:

`/announcements` — Cards feed: priority badge, title, content preview, posted by, date. URGENT announcements pinned at top with red border.

`/helpdesk` — Employee: my tickets list + Raise Ticket button

`/helpdesk/new` — Form: category, priority, title, description

`/helpdesk/manage` — HR/Admin: all tickets table, filter by status/category/priority, assign to self button, resolve button

MODULE 11: REPORTS & ANALYTICS

API Endpoints:

```
GET  /api/reports/attendance   → attendance summary (monthly, by employee)
GET  /api/reports/payroll      → payroll summary by month, export CSV
GET  /api/reports/leave        → leave utilization by type and employee
GET  /api/reports/headcount    → employee count stats (active, inactive, by role)
GET  /api/reports/tasks        → task completion rate by employee
GET  /api/reports/projects     → project status overview
```

UI Pages:

`/reports` — HR/Admin only. Dashboard with:
Headcount: Bar chart (by role), Pie chart (by status)
Attendance: Line chart (monthly present % trend), Bar chart (department-wise)
Leave: Bar chart (leave type utilization), Table (top leave takers)
Payroll: Bar chart (monthly total payroll cost), Line chart (net salary trend)
Tasks: Bar chart (tasks by status), Table (completion rate by employee)
Projects: Pie chart (by status)
All charts use Recharts. Each section has a "Export CSV" button.

MODULE 12: DASHBOARDS (Role-Specific)

Admin Dashboard (`/dashboard`):
Stats row: Total Employees, Present Today, On Leave Today, Open Tickets, Active Projects
Charts: Headcount by role (pie), Monthly payroll trend (line), Attendance this week (bar)
Tables: Recent employee additions, Pending leave approvals, Open helpdesk tickets
Quick actions: Add Employee, Run Payroll, Post Announcement

HR Dashboard (`/dashboard`):
Stats: Pending leave approvals, Payroll due this month, Open job postings, Unresolved tickets
Charts: Leave utilization (bar), Attendance rate this month (line)
Tables: Pending leave requests, Recent applicants, Upcoming interviews

Employee Dashboard (`/dashboard`):
Greeting: "Good Morning, [Name]" with current date
My Today: Punch In/Out widget, today's attendance status
Leave Balance: Mini cards per leave type
My Tasks: Count by status (To Do / In Progress / Done)
Upcoming: Tasks due this week, Announcements feed
Latest Payslip: Quick summary card with Download button

Intern Dashboard (`/dashboard`):
Same as Employee but no payslip section — shows Stipend card instead
Limited to own tasks and projects

UI / UX DESIGN SYSTEM

Color Palette

```
Primary:   #1E3A5F  (deep navy — cybersecurity trust)
Accent:    #00C2FF  (electric cyan — tech/security aesthetic)
Success:   #22C55E
Warning:   #F59E0B
Danger:    #EF4444
Background: #F8FAFC  (light mode), #0F172A (dark mode)
Card:      #FFFFFF  (light), #1E293B (dark)
Text:      #1E293B  (light), #F1F5F9 (dark)
Border:    #E2E8F0  (light), #334155 (dark)
```

Typography

Display/Headings: **Inter** (700, 600)
Body: **Inter** (400, 500)
Monospace (IDs, codes): **JetBrains Mono**

Layout

Sidebar: fixed left, 260px wide, collapsible to 64px icon-only on mobile
Sidebar items: grouped by module with icons (use Lucide React icons throughout)
Topbar: 64px height, shows current page title, notification bell with badge, user avatar dropdown
Content area: scrollable, max-width 1400px, padding 24px
Mobile: sidebar becomes bottom drawer, all tables become cards stacked

Sidebar Navigation Structure

```
📊 Dashboard
👥 Employees        (HR/Admin only)
🕐 Attendance
🏖️ Leave
💰 Payroll
✅ Tasks
📁 Projects
🤝 Clients          (HR/Admin only)
🎯 Recruitment      (HR/Admin only)
⭐ Performance
🖥️ Assets           (HR/Admin only)
🧾 Expenses
📢 Announcements
🎫 Helpdesk
📈 Reports          (HR/Admin only)
⚙️ Settings         (Admin only)
```

Shared Components to build

`<DataTable>` — sortable, filterable, paginated table with row selection and export
`<StatCard>` — icon + label + value + optional trend arrow + colour variant
`<PageHeader>` — title + breadcrumb + action button(s)
`<StatusBadge>` — pill badge with colour per status enum
`<ConfirmModal>` — reusable confirmation dialog
`<EmptyState>` — illustration + message + CTA for empty lists
`<Avatar>` — circular with initials fallback
`<NotificationDropdown>` — bell icon with unread count, dropdown list, mark-all-read
`<FileUpload>` — drag-and-drop area with file type/size validation

REAL-TIME NOTIFICATIONS (Socket.io)

When these events happen, emit a Socket.io event to the relevant user(s), and save a Notification row to the DB:

Event	Who gets notified
Leave request submitted	HR + Admin
Leave approved/rejected	The applicant
New task assigned	Assignee
Task status changed	Task creator
New announcement posted	All users in target roles
Payroll generated	All employees
Helpdesk ticket assigned	Assigned agent
Ticket resolved	Ticket raiser
New applicant added	HR + Admin
Expense claim submitted	HR + Admin
Expense approved/rejected	Claimer

Frontend: Notification bell shows unread count badge. Clicking opens a dropdown list with notification message + timestamp + link to relevant page. "Mark all as read" button.

SETTINGS MODULE (`/settings` — Admin only)

Company Settings: Company name, logo upload, address, GSTIN, financial year start month
Leave Policy: Edit leave types (name, days, carry-forward rules)
Holiday Calendar: Add/edit/delete public holidays for the year
Payroll Settings: Default PF%, ESI%, TDS%, payroll processing date

User Management: List all users, reset password, change role, deactivate
Audit Log: Read-only log of all Admin/HR actions with timestamp and actor

SECURITY REQUIREMENTS

All routes protected by JWT middleware
Role-based access enforced on BOTH frontend (route guards) and backend (middleware)
Passwords hashed with bcryptjs (saltRounds: 12)
httpOnly cookie for refresh token (not accessible via JS)
Rate limiting: 100 requests/15min per IP on auth routes
Helmet.js for security headers
CORS configured to allow only the frontend origin
File uploads: validate file type (pdf, jpg, png only), max size 5MB
All user inputs sanitized and validated (Zod on backend)
SQL injection impossible via Prisma parameterized queries

ENVIRONMENT VARIABLES

env

```
# Backend (.env)
DATABASE_URL="postgresql://user:password@localhost:5432/prsecurity_hrms"
JWT_SECRET="your-super-secret-jwt-key-min-32-chars"
JWT_REFRESH_SECRET="your-refresh-secret-key-min-32-chars"
JWT_EXPIRES_IN="15m"
JWT_REFRESH_EXPIRES_IN="7d"
PORT=5000
CLIENT_URL="http://localhost:5173"
NODE_ENV="development"

# Email (for password reset)
SMTP_HOST="smtp.gmail.com"
SMTP_PORT=587
SMTP_USER="noreply@prsecurity.in"
SMTP_PASS="your-app-password"

# Frontend (.env)
VITE_API_URL="http://localhost:5000/api"
VITE_SOCKET_URL="http://localhost:5000"
```

DOCKER SETUP

Provide a `docker-compose.yml` that runs:
PostgreSQL 15 on port 5432 with volume for persistence
pgAdmin on port 5050 for DB management

SEED DATA

Seed the database with:
Default Admin account: `admin@prsecurity.in` / `Admin@1234`
All 8 leave types as specified
Sample holiday calendar for 2025 (national holidays of India)
Sample leave balances for the admin for current year
One sample HR account: `hr@prsecurity.in` / `Hr@1234`

README.md MUST INCLUDE

Prerequisites (Node 18+, PostgreSQL, pnpm/npm)
Clone and setup steps
Docker setup for DB
Environment variable configuration
`npx prisma migrate dev` command
`npx prisma db seed` command
Start frontend and backend commands
Default login credentials
Folder structure explanation
API documentation link or inline for all major endpoints

FINAL REQUIREMENTS & CONSTRAINTS

Every single page listed above must be built — no placeholders, no "coming soon" pages
All API endpoints must be fully implemented — no mock data in production code
Mobile responsive — every page works correctly on screens 375px and above
No data loss on restart — all state in PostgreSQL, no in-memory storage used for persistence
TypeScript strict mode — no `any` types; all entities fully typed
Error handling — all API errors return structured JSON `{ success: false, message: string, errors?: any }` with appropriate HTTP status codes. Frontend shows toast notifications for all success/error states.
Loading states — every data fetch shows a skeleton loader or spinner, never a blank flash
Empty states — every list/table shows a meaningful empty state component when no data exists
All amounts in INR ₹ — use `Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR' })` everywhere
Date format — DD MMM YYYY throughout the UI (e.g. 15 Aug 2025)
Dark mode support — Tailwind dark: classes used throughout, toggle in user profile dropdown
Accessibility — all interactive elements have aria-labels, keyboard navigable, focus rings visible
Code quality — ESLint + Prettier configured, consistent naming conventions, no unused imports
Git-ready — provide `.gitignore` for both frontend and backend

DELIVERY FORMAT

Output the complete project as follows:
Start with `docker-compose.yml`
Then `backend/prisma/schema.prisma` (complete schema)
Then `backend/prisma/seed.ts` (complete seed)

Then all backend files in order: app.ts → middleware → routes → controllers → services
Then frontend files: App.tsx → routes → store → api → components → pages (in module order)
End with `README.md`
For each file, output the **complete file content** — never use ellipsis (`...`) or "rest of file remains the same". Every file must be production-ready and complete.

Company: PRSECURITY CONSULTANCY & SERVICES | Location: Surat, India | Currency: INR ₹ | Scale: ~30 users